

Staying technical as an engineering manager

The importance of staying technical, and activities to do at work, and outside work. Also: why Amazon's Deep Dive principle helps stay technical.



GERGELY OROSZ

MAY 2, 2023 • PAID



107



1



9

Share



Q: “As an engineering manager, how can I stay technical when I no longer do hands-on work?”

In a previous issue, we've covered [ways to stay hands-on as an EM or tech lead](#), in which we go through the hands-on work you can do.

In today's issue, we look into ways to stay technical when you are no longer 'at the coalface' doing hands-on work, or doing very little of it. Today, we cover:

1. The importance of staying technical
2. Activities to do at work
3. Activities to do outside of work hours
4. Staying technical when you're distant from the work
5. Amazon's "Deep Dive" principle
6. Advice from managers on staying technical

1. The importance of staying technical

It's beneficial to stay involved with what your team's doing, as you can course correct when things go astray. At least, that's the theory. The challenge is that it's hard to be

properly engaged without understanding exactly what colleagues are doing. And in order to do that, you need to keep up to date technically.

For example, you might notice more bugs than usual are making it to production. If you're a non-technical manager, you'd likely rely on your experienced engineers to explain why. In contrast, as a technical manager you can personally review recent code changes and detect untested changes which slipped through, then give the relevant engineer feedback. For example, you might observe an engineer is inexperienced with testing. A technical manager can mentor them personally, or allocate a mentor to help the engineer improve, or support them in other ways to plug the skills gap.

By staying both technical and involved, you can also decide when it makes sense for you to be hands on; for example, stepping in to resolve an outage, or resolving a deadlock in an architecture decision.

Staying technical is a must for any manager who wants the option of being hands on. If you stop being technical, then you'll struggle more to be hands on in future, even if you want to.

Say for example that after becoming a manager, you focus purely on people and stakeholder management. You trust your team and so no longer pay attention to code changes or architecture reviews. One day, your team decides to migrate to a new framework, but something goes awry and causes an outage. A non-technical manager won't know enough about the framework or the migration choice to understand why it happened or how to fix it. Even if you want to, you cannot be hands on in this situation without familiarizing yourself with the migration and the new framework.

Staying technical as an engineering manager brings a host of benefits:

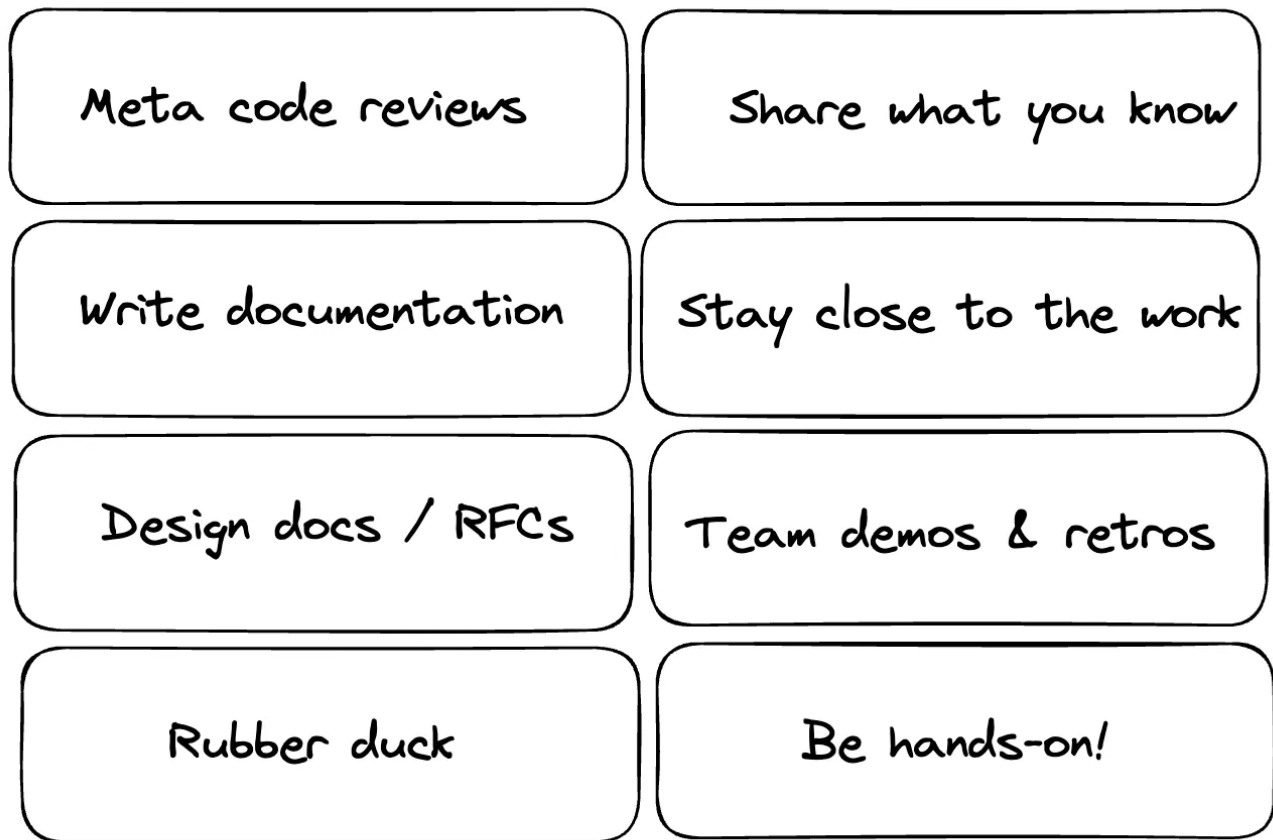
- *The choice of when to get involved.* You can choose to be hands on when the situation warrants.
- *You keep learning.* This keeps the job interesting!
- *Job security.* An engineering leader who stops being technical is a lot less employable than one who is technical.

- *Compensation.* Technical engineering leaders who can be hands-on when needed, usually command better compensation than peers without this skill. Back in 2012, Twitter's SVP Christopher Fry made \$10.3M in compensation, more than the company's CFO and COO. At the time, Reuters interviewed former colleagues of his, who [said](#) he fitted the bill of a "10x engineer," while also being a leader with a good track record. We don't know all the details, but it reads like that Chris was both an outstanding leader and an outstanding engineer.
- *Personal satisfaction.* If you're an engineer at heart, you enjoy understanding how things work, and feel frustrated when you don't. Staying technical supports this kind of satisfaction, while your frustrations are reserved for non-technical topics.

Of course, the challenge is balancing staying technical and being hands on, with doing all that other work that's necessary for your team to operate well.

2. Activities to do at work

The good news is that although you might be able to dedicate less time than you'd like to hands-on activities, there are plenty of ways to stay technical and involved in what your team does.



pragmaticengineer.com

Activities to stay technical, as an engineering manager or lead

Meta code reviews. A code review is when you check a code change, leave comments, and approve or reject the change. A meta code review is on one level, abstract: you simply read through the reviews, usually the completed ones.

But why do meta code reviews and not regular code reviews? Usually, it's down to the amount of time you can invest and how frequently they happen.

If you can set aside only a small amount of time each week for code reviews, then you'll likely miss reviews at the team level. In this case, it might be worth going through all the code reviews that happened *before*, make notes as you go, and discuss questions or observations later.

When I was an engineering manager, I'd end most weeks by reading through code reviews by my team, so I understood what was happening and could identify areas I didn't understand.

This activity is also helpful for getting a feel for team dynamics. Meta code reviews can catch conflicts between colleagues early, and signal when intra-team dynamics change for the better or worse. Code reviews involving lots of back-and-forth, or which are overly critical and nit-picky, can be signals that something is at play between the people involved. It might be that an engineer misunderstands the code, or that there's a brewing conflict, or something else. By doing meta code reviews as an engineering manager, you can spot such things early and work out how to address them. Without this activity, you might only notice there's a problem when an engineer tells you.

Write documentation. This is something most teams – and engineers – invest much less time and effort in than they'd like. This comes with a downside as a lack of documentation can delay progress when information is urgently required.

You might not have much time to be hands-on with coding, but as an engineering manager you can help the team and stay very much involved by writing up missing documentation.

However, in order to document something, you must first understand it. So if a new system the team built lacks information about how to onboard to it: you learn how to do it by documenting this process. The same goes for recording which systems the team owns in writing or a diagram, how they connect, and what the migration plans and timelines are.

Being proactive about documentation – and encouraging others to update details and amend documents – can also set a good example to colleagues and may motivate some engineers to do it, too. Of course, don't do this primarily as an example, but because it helps both the team and yourself to stay technical and involved.

Read design docs / RFCs / ERDs / ARDs. Even if you don't make comments, examining design documents keeps you up to date. As a manager, there's scant reason not to read every design document your team creates. It's also helpful to skim-read design documents by other teams, be they engineering or product-focused.

We cover more about design docs in [Engineering planning with RFCs, design documents and ADRs](#).

Act as a rubber duck. If you notice an engineer on your team is stuck, offer to be a “rubber duck.” This means being someone they can talk the problem through with. You can do this even if you have little technical or domain knowledge, as you’re helping the engineer solve the problem themselves.

Of course, as a manager, there’s a fair chance you have plenty of domain knowledge and also some technical knowledge, so you might be able to help in more active ways. Even so, just helping an engineer to work mentally through a problem and reach a solution can be helpful, not just for the engineer but also for you, as you learn more about it.

Share your knowledge and teach. One of the best ways to understand something in depth is to teach it. Once again, this is because you need to understand it in order to teach it. Therefore, volunteer to teach things and share knowledge; it will deepen your own understanding. Volunteer to teach or share technical topics in order to grow technically and better understand a domain.

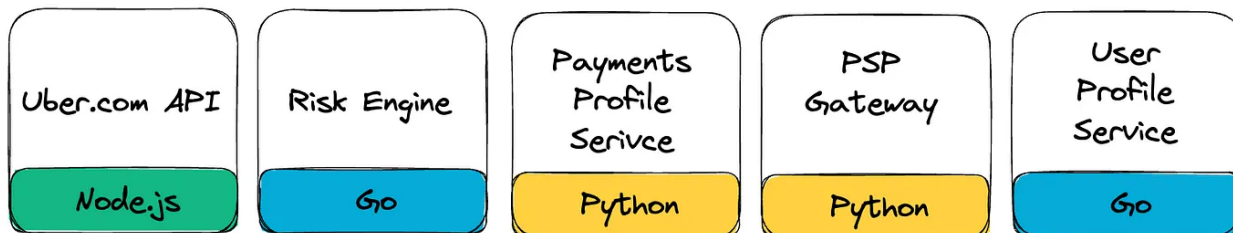
When I was at Uber, I didn’t fully appreciate how the Payments systems worked beyond my team and others around us. So I volunteered to put together an onboarding session for new joiners about precisely this topic, featuring an overview of all Payments systems and spanning about 10 teams. In order to teach this topic, I drew up diagrams, then asked engineers in each team for help in filling gaps on workflows and technologies. When I first delivered this session, there were questions asked I did not have answers to, so I went back to find them. Had I not given this onboarding session about the Payments systems, I’d have understood these systems less well.

Volunteer to participate in or lead internal training on technical topics. If your team is in the habit of holding brown bag sessions – short knowledge-sharing sessions during lunch – then volunteer to hold one about a topic you’ve always wanted to learn more about. When you do, you create the internal motivation to learn the topic well enough to explain it to others!

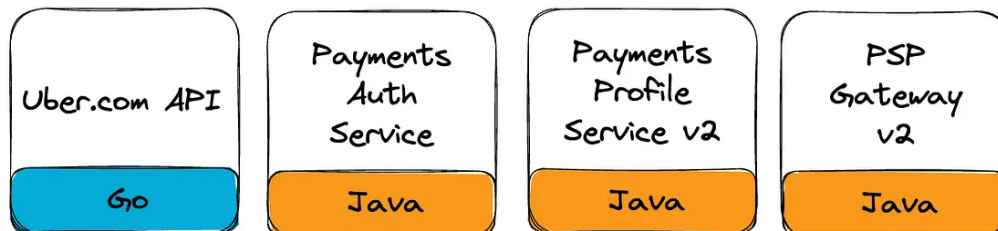
Talking at a meetup or a conference is also a great way to force yourself to both understand and explain a topic. I’ve attended meetups and conferences as a speaker, and preparing for them helped crystalize my understanding. For example, when I [talked](#)

[about Uber's payments integration platform](#) at a tech meetup in San Francisco, I did plenty of research to ensure I correctly understood every relevant system involved in the payments integration flow:

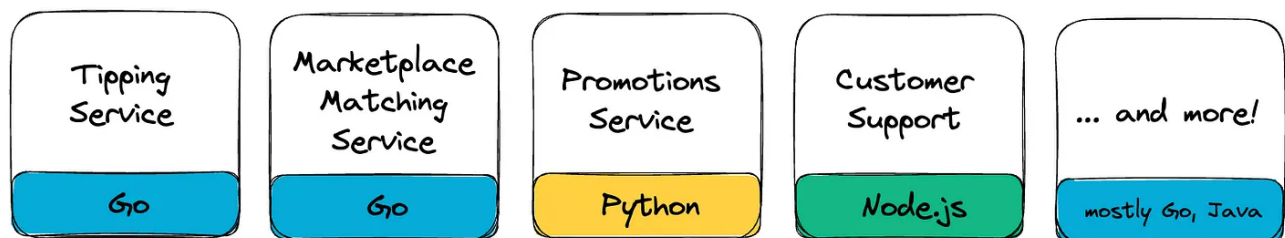
Core systems ("current" ones in 2018)



Nextgen systems (WIP)



Other systems with payments business logic



pragmaticengineer.com

A recreation of a slide I used during a talk in 2018 about Uber's payment systems. While preparing, I had to familiarize myself with systems I was unacquainted with, so I could explain what they did and technology choices behind them. [See the full talk here.](#)

Write a technical post for your company blog or personal blog. Explaining concepts in writing can feel more challenging, especially for a wide audience. Then again, until you can explain things, can you really claim to understand them? *I previously shared advice on [Becoming a better writer.](#)*

Stay close to where the work happens. Although you might have less time for being hands on, there's several ways you can remain close to the action:

- *Local environment.* Have the local environment set up and exercise it. Even if you don't write production code, have the local environments ready to use, and in a way that you can run local builds, start local clients, and run tests. This is especially useful if you will later have little time to fix bugs, or debug.
- *Logs.* Know where to find and how to read important logs. Where are the production logs for errors and warnings, the build logs, the deploy logs? Are these bookmarked?
- *Systems' health.* Regularly look through errors and systems health metrics. Even if your team does not have an operations review, keep an eye on systems' health metrics. Ask yourself if there should be a regular review.
- *Customer feedback.* Check the pulse of customer support tickets. Make sure you have visibility of – or are notified about – pertinent comments, complaints and problems. In many teams, product managers or operations people receive these inbound messages, but not engineering managers. Even so, you should have at least some visibility of these inbounds, in order to be aware of sudden upticks and strange or important reports which arrive.
- *Build tools.* While you might have less time to write production code, consider building small tools which help your team or automate tedious tasks. Many technically involved engineering managers I've observed do exactly this: they create scripts which automate tedious processes like creating onboarding tasks for new joiners, parsing debug logs better, and so on. Building helpful tools is almost like a hobby, just one that's useful at work.

If you're wondering, "should I really carve out time for reviewing logs and systems' health metrics?," here's [some inspiration](#) from the founder and CEO of Pebble Finance, a seed-stage startup with 7 engineers:

**Justin Whitehead**

@whitej125



Simple routines for a start. Each morning, with a coffee in hand, I find at least one sentry I can investigate/solve before others wake up.

Team demos and retrospectives. The less hands on you are, the less reason there is for not attending your team's demo sessions, where they present their work and explain how it was done. It can also be beneficial to attend demos by other teams with whom you collaborate.

Retrospectives are useful for getting the pulse of what's on the team's mind. This isn't really a way of staying technical, but of being involved and engaged with your team's wins and challenges.

If you can be hands on, be so. We cover activities you can do to stay hands-on in [Staying hands-on as an engineering manager or tech lead](#). Suggestions include:

- *Participate in design docs / RFCs / ERDs / ADRs.* Don't just read them: challenge them and add your thoughts.
- *Code reviews.* Get involved, though pay attention to not blocking them.
- *Operational reviews.* Facilitate these and dive deep into issues surfaced.
- *Go oncall.* Though disruptive, this forces you to get up to speed with monitoring and debugging systems.
- *Stay active during outages and incident reviews: Ensure you understand what happened and why, and what caused an incident.*
- *Write code.* If you can, work on low-priority tasks, prototype, pair program, join hackathons or "fixit weeks."
- *Beware of taking too much from others. Be mindful to not "snatch" the best work from engineers on your team when being hands-on.*

3. Activities to do outside of work hours

Stay up to date with technology. The tech landscape always changes: new tools and approaches appear and existing ones become outdated. On the backend, good examples of this are languages like Scala, Rust or Elixir gaining popularity, or approaches like serverless gaining momentum. The same is happening on the frontend: micro-frontends, frameworks like React or Next.js, within mobile and cross-platform frameworks. We've not even talked about innovation in [data engineering](#), machine learning and AI.

So how do you keep up with all this progress?

Side projects outside of work, or prototyping at work, are two of the surest approaches. Most engineering leaders whom I consider as very technical have played with new technologies outside of work, building proof-of-concept projects to try out new frameworks, languages, platforms and approaches. They often enthusiastically share interesting learnings at work and sometimes inspire engineers to examine those technologies.

Ahmad Awais is a VP of Development Relations at RapidAPI, and [swears](#) by this approach:



Ahmad Awais
@MrAhmadAwais



Side projects are my go to. Have been coding every day for almost half a decade. Even though as a VP I don't need to now.

Of course, you can keep up with developments in many ways: reading blogs, technical articles, books, videos, and taking video courses. Attending conferences is a great way to learn about modern approaches and also meet like-minded people. Find the formats which work best for you and fit your time and expense budgets.

And of course by reading this issue you're already keeping up with approaches and changes across the tech industry!

Keep a finger on the industry's pulse. Which new technologies are gaining momentum and could be worth checking out, which trends are worth paying attention to? There are several ways you can keep track, including:

Newsletters about new developments. There is an ever-growing collection of newsletters which scan online resources like engineering blogs, videos and talks, and collate interesting takeaways about what's going on. Many publish weekly and you can often find them by googling your domain of interest and discovering relevant newsletters. Here's a few for inspiration:

- Frontend: [JavaScript Weekly](#), [This Week in React](#), [WDRL](#) (daily)
- Native mobile: [iOS Dev Weekly](#), [Android Weekly](#)
- Backend related: [This Week in Rust](#), [Golang Weekly](#), [Java Weekly](#), [Off By One](#) (Serverless), [LLVM Weekly](#), [Laravel News](#) (PHP)
- Data: [Data Engineering Weekly](#)
- Security: [tl;dr sec](#)
- Generic software engineering: [The Pointer](#) ("a reading club for developers"), [Programming Digest](#), [TLDR](#) (a daily newsletter), [Bytebytego](#) newsletter (systems design-related), [Technically](#) newsletter, [Architecture notes](#) (systems design-related)

Podcasts.

- Frontend: [JS Party](#), [DevMode.fm](#), [Syntax.fm](#), [Django Chat](#)
- Native mobile: [Fragmented](#) (Android), [Android Developers Backstage](#)
- Backend related: [Go Time](#), [Rustacean Station](#) (Rust), [Talk Python to Me](#), [CppCast](#) (C++), [PHP Roundtable](#), [PHPUgly](#), [.NET rocks!](#), [Hashicast](#)
- Data: [The Data Engineering Show](#), [Data Engineering Podcast](#)
- Security: [Darknet Diaries](#), [Security Now](#), [Security, Cryptography, Whatever](#)
- Generic software engineering: [Changelog](#), [Corecursive](#), [Software Engineering Daily](#), [Hanselminutes](#), [Ping](#) (the workings of the internet) [Software Engineering Radio](#), [Ship it!](#), [The New Stack](#)

Educational resources. Some prefer more structured resources such as educational books and tutorials. Some popular locations for these:

- Books: [O'Reilly's learning library](#) (access to hundreds of technical books and some courses.) Other publishers of high-quality technical titles include [Manning Publications](#) and [The Pragmatic Bookshelf](#).
- Online learning: [Frontend Masters](#) (frontend technologies), [Kodeco](#) (iOS, Android, Flutter), [Pluralsight](#) (a large range of technologies.)
- YouTube channels: [Continuous Delivery](#) by Dave Farley (CD, DevOps, TDD and software engineering), [Code Opinion](#) (software architecture), [C++ Weekly](#) with Jason Turner, [Philipp Lackner](#) (Android), [Nick Chapsas](#) (C#), [PHP Annotated](#)

[*Browse the above resources in a standalone document.*](#)

Learn new concepts by getting hands on. When you encounter an interesting technology or concept that might be worth exploring, play around with it by trying to build a prototype or a real project. Alternatively, document your learnings, or share with people around you.

For example, VP of Engineering and fellow Substack writer **Dustin Diaz** built the website [Still Hiring](#) on the side, building it from idea to launch within five days, using HTML and JQuery on the frontend, NodeJS / ExpressJS / Pug, Redis, PostgreSQL on the backend, and Heroku and AWS S3 for hosting. See a longer list [of technologies he used here](#).

Similarly, Will Larson, CTO of Calm, [wrote a scraper for Twitter](#) and shared the code on GitHub. He also [played around with VPN service Tailscale](#), documenting his learnings.

Do code katas. Finally, even if you don't have the time or opportunity to code at work, you can do some code katas, which are standalone coding exercises. Plenty of people swear by code katas as a way to stay fresh and exercise their skill with a language, or to pick up a new programming language. The nice thing about code katas is you can do them at your leisure and don't need to spend much time setting up the environment. Here's [a collection of code katas](#). Give them a go before passing judgment; I've done these to become more hands on with a new language.

4. Staying technical when you're distant from the work

It's easy to stay technical when you're hands on. When you're less hands on, it takes more purposeful effort. The higher you move up the career ladder and the farther away you get from where coding happens, the harder it is to stay technical.

This tends to happen to software engineers in a couple of ways:

- Promoted to a management position and then promoted to manage managers. While being hands-on is relatively easy as a line manager, it's more challenging as a manager of managers.
- Moving sideways to become a Technical Program Manager (TPM) or a Product Manager. Most activities in these roles are about strategy, working across teams and collaborating with technical and business colleagues.

Staying technical seems much easier for people who *want* to be involved in the details, even when their day-to-day makes it hard to be hands-on. Some of the most technical engineering directors I've seen insisted upon understanding the detail of how systems work across their teams.

They were directors and VPs who joined a company and soon sat down for deep dives with engineers, requesting they whiteboard how systems work and questioning design choices until they understood why decisions were made. These directors were not only technical, but they were hands on when needed.

5. Amazon's "Dive Deep" principle

I'd been thinking about how to best summarize traits that technical leaders have in common, when **Rex Morgan**, engineering manager at Amazon, [shared](#) an interesting observation about staying technical:

"I prefer Amazon's leadership principle of "Dive Deep" - leaders operate at all levels, stay connected to the details. The important question for a manager is "are they

effective" which is impossible without diving deep, but what that looks like depends on the context!"

So what is this "Dive Deep" principle? It's one of Amazon's 16 leadership principles, which we cover at a high-level in [Inside Amazon's engineering culture](#). Here's how this principle is defined:

"Leaders operate at all levels, stay connected to the details, audit frequently, and are skeptical when metrics and anecdotes differ. No task is beneath them."

I really like this definition because it covers staying technical ("stay connected to the details") and being involved ("audit frequently,") and being hands on when needed ("no task is beneath them.") But that's not all. This principle also makes clear that the expectation is broad ("operate at all levels,") while leaving implementation to the leader.

Of course, this is a demanding principle. But at root it really asks a leader to stay up to date, to inspect frequently, and to be hands-on *only* when needed.

One of the best explanations of the principle comes from Dave Anderson, who spent 6 years at Amazon – including as director of technology – and who was later CTO at the Bezos Academy. Dave writes the weekly [Scarlet Ink](#), a paid newsletter about leadership. One thing Amazon's leadership principle does not cover is technical references and skills. Dave walks through the principle with practical examples in his article [A guide to the dive deep leadership principle](#). Here's a few interesting quotes from it.

On operating at all levels:

"The challenge is that you can't always do senior-level work. The pointy haired boss in Dilbert is a joke because he doesn't actually understand how his teams operate. He is the boss, but not a leader. A leader interacts with everyone on their team, not just with those who are also senior."

On staying connected to details:

"Due to their visibility into the larger picture, leaders create value through paying attention to the details."

As one example, I remember attending a meeting for a team which managed a component on Seller Central. They spoke about how their next project was to re-write some components to reduce latency (component load time).

I asked why the latency project was high-priority. They said that the page their component was on was too slow, and users were complaining. At the time, I ran the latency program for Marketplace, and I had personal knowledge of how many of these pages were constructed.

I asked that team to please confirm that speeding up their component would actually speed up the page load time. Because I knew that it was a different component on the page which was the bottleneck for load times. Shortly afterwards, they explained that they had discovered that speeding up their component would not, in fact, improve user experience, so they were updating their roadmap."

On being skeptical when metrics and anecdotes diverge:

"Strangely, 90% of the time, Jeff knew when to audit. Black magic, I assume. He'd find the one strange edge case and question it. We'd show ourselves to be blind, that he'd figured out a complex and detailed edge case from a single non-specific customer email.

In general, if you have metrics, you should audit them. You should verify that they're telling you the truth, and perhaps more importantly, your assumptions behind those metrics."

In the article, Dave gives examples of all elements of the principle, and highlights just *how* carefully and precisely worded Amazon's principles are. [Read the full article here.](#)

The concept of the "deep dive" very much resonates with me. Reflecting on deeply technical engineering leaders, all shared the trait of being willing to dive deep into domains when they felt the need, and thereby filled gaps in their knowledge and stayed technical.

6. Advice from other managers

[Mark Tsimelzon](#) is a director of engineering at WhatsApp. He's also written advice [on staying technical as an engineering manager](#):

1. **Accept that being a manager is very different.** "As an engineer, you have the luxury of understanding everything you need to understand: the code base, the libraries and platforms you use, the tools. (...) In contrast, as a manager, you don't have this luxury of understanding everything."
2. **Build mental models.** "The next best thing is to start building mental models in your head of how things work. These models will not cover every little detail, and will not go all the way down to very specific interfaces, so they will necessarily be shallow and fuzzy. But a shallow fuzzy model is still better than no model at all."
3. **Focus on the boundary between product and engineering issues.** "Each one of your engineers may be an expert on the internals of a particular part of the product, but there are usually interesting challenges where they all come together."
4. **Focus on metrics.** "Some of the details of what a given piece of data means, and where it comes from, can be very technically involved."
5. **Invest in understanding fundamental technologies.** "Every few years large technology shifts appear. Cloud Computing. NoSQL data stores. Big Data. Microservices. A mega-shift such as Machine Learning. It's good for you to understand these issues, and to get some hands-on experience with them as they appear."
6. **Do write some code if you can.** "The farther you are from coding, the less you'll understand the development process, the environment, the tools, and the challenges your team faces. Writing some production code here and there keeps you close to these areas."
7. **Figure out how you learn best.** "Given the limited amount of time, optimize your technical learning."

Mark examines each point in greater detail [in his original post](#). I can also recommend [following Mark on LinkedIn](#), where he shares advice.

[Nivia Henry](#) is a director of engineering at Spotify. Her name might be familiar to readers from when she shared learnings of her career in the issue [From a non-traditional background to Director of Software Engineering at Spotify](#). Answering the question of what she does to stay technical, Nivia says:

"Let me start by what 'technical' means in my context, as a director, who manages senior managers. Nobody expects me to push production level features at work; I don't have enough of the daily context and in fact it's a red flag for many reasons.

Technical for my level means knowing my tech stack enough to know the gaps and where to invest. It means being able to advocate upward for investments to my architecture, as well as being able to provide direction to my teams about how to build and scale on it.

For example, it is appropriate for me to produce RFCs on a particular domain's design, which I do regularly. My last one was the design of a reporting system that needed both internal and external APIs.

I also spend a good portion formulating technical strategies to bring the product and business strategies to life. I also periodically facilitate discussions on particular implementations and tradeoffs. In those, I remind the teams of our system gaps and our technical strategy.

Those are more appropriate uses of my time and a better way to stay technical because it gives my talented engineers room to contribute.

In the cases when I need to directly write code as an ego booster, I have hacks and side projects to do that safely."

Shopify CTO Allan Leinwand has also [shared his insights](#) on staying technical:

1. **Be a voracious leader.** "You need to be able to consume data outside of meetings, and you have to learn to absorb information quickly. Have a question about how something works in the organization? Go read the code. Go read the documentation. I spend a lot of time reading on Github. I read commits, read

through the documentation, look through test results, and see if we're releasing things properly and at the right time."

2. **Code as a hobby.** "I code on side projects that aren't going to necessarily end up in a Shopify production repo. As a technical leader, you need to make coding part of your fun, part of how you gain understanding of the developer toolset."
3. **Participate in hackathons.** "I'm not often hands-on coding, but I join various projects and work with teams trying to figure out how we can build the best products and innovations for our merchants."
4. **Hire well.** "Good technical people can teach you things faster than you can read about them. When I hire, I look for people who are super technical."
5. **Create a team of tech advisors.** "I find having a team of tech advisors immensely useful. It's another way I stay technical as a CTO—essentially crowdsourcing on technical topics."
6. **Clear out bugs and backlog.** "Getting ahead of bugs and remediation items in the backlog and then allocating time for teams to work on innovation and tech debt is absolutely critical in allowing you to focus on being a great technical leader."

Allan goes into more detail about each point in the article [Six tips for staying technical as a CTO](#).

Takeaways

When you work with software engineers every day, for example as a tech lead manager or a first-line engineering manager, it's easier to stay technical and occasionally be hands-on. But as a manager of managers, you lose many of these opportunities and it becomes harder to stay technical.

This issue is a follow-up to [Staying hands-on as an engineering manager or tech lead](#). We've covered approaches for staying technical when this is harder to do.

- Meta code reviews: reading code reviews, even when you don't actively participate in them.
- Write documentation: even if you don't have time for coding.

- Consult design documents / RFCs / ERDs: even when you have less time to actively participate in them.
- Act as a rubber duck for engineers.
- Share your knowledge and teach.
- Stay close to where the work happens.
- Team demos and retrospectives; be present.
- Stay up to date with technology: do side projects, keep a pulse on the industry.

Here are [resources \(newsletters, podcasts, educational sites\) that can help do so](#).

The “Dive Deep” principle at Amazon is surprisingly relevant to any leader who wants to stay technical while helping their team get things done. Diving deep is more than just staying technical: it’s about thoroughly educating yourself and being hands on when needed.

Being technical and having the ability to be hands on are linked, so don’t try separating them. Every now and then, roll up your sleeves to help your team, even if the benefit is marginal. Without hands-on work, your technical chops will rust and you’ll be at risk of becoming that leader with no clue of the technical details.

Remember your primary responsibilities, while staying technical. As a manager or leader, your role is to ensure your team is as efficient as possible and operates better than without you.

Being technical with the ability to get hands on will help you spot issues early, give feedback, and verify the quality of work from an engineering perspective. Being strong technically means you can help your team at a tactical level, not just strategically.

However, it doesn’t matter how efficient your team is if they work on the wrong thing, do throwaway projects, or get blocked by factors beyond their control. The challenge to staying technical is that strategic work unavoidably removes you from the fine detail. If you ensure your team is working on the *right* thing and you better understand what *right* means for your team, group and company, then you’ll naturally be distanced from where the “action” happens.

Every engineering leader I've talked with has a slightly different approach for staying technical. However, there's a few methods which are popular and worth repeating:

Coding. Make time for this, every now and then. It's better if it happens in your workplace's codebase, but if not possible, doing it on the side helps keep your knowledge of this core activity fresh.

Metrics, outages, backlogs. Stay involved in the "boring" and "non-desirable" parts of running an engineering team. Look into metrics and logs, understand why outages happen, and be aware of what makes it to the backlog, and debt that might need to be addressed.

Keep learning and trying out new concepts. The tech landscape changes fast. Right now, an AI wave is headed for the data world, micro frontends and TypeScript are becoming more popular in frontend, and likewise technologies like serverless and Rust on the backend, Flutter and React Native on the mobile side, to name just a few.

Choose a couple of areas of interest and dive deep into them, so you keep pace with technologies' promise and capabilities, and can formulate informed, first-hand opinions.

What has worked for you in staying technical? Share in a comment below:

As a heads up: I'm wrapping up my spring break, [as per my holiday schedule](#). This means that this week there is no Thursday, The Scoop issue. I'll be back on the usual schedule from next week!



107 Likes · 9 Restacks

1 Comment



Write a comment...



Joshua Jun 5

I don't always get to read these articles immediately and I usually read on my phone where it's handy to 'open online' to avoid losing my place if I need to deal with another email. Why do I continually get this message for 'older' articles such as this (below) when I can access the page just fine from a Google search result?

> Substack is experiencing technical problems. We are working to restore service. Thank you for your patience.

>

> You can find up-to-date information on technical outages at status.substack.com

♡ LIKE 💬 REPLY ...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing